

Session 05 — Project Guide

TaskFlow: React + TypeScript Foundations

This guide documents the official **Session 05 End** state of TaskFlow. Everything in it comes from the real project files — nothing is invented, and nothing describes the homework. Keep this guide. You can read it at home, several days later, and rebuild the whole mental model from it alone.

1. The Session Story — What Actually Changed Today

Look back at the road so far:

- **Sessions 1–3** — HTML, CSS, and JavaScript. You built the page, made it interactive, then made it load real data from an API.
- **Sessions 4/4B/4C** — TypeScript. Same app, same logic, same features — now written in a language that checks the *kinds* of values you're working with before the browser ever sees the code.
- **Session 5 — today**. We are **not** throwing TypeScript away. Every component you write today is still TypeScript underneath (`.tsx` files). What changes today is **how the UI itself gets built and put on the screen**.

Here is the actual shift, in the same "before → after" shape every earlier session used:

BEFORE (Session 4C):

data changes

- you find the right element yourself (`document.querySelector`)
- you change it yourself (`.textContent`, `.innerHTML`)
- you call a render function yourself, by hand

NOW (Session 5):

you describe what the UI should look like, using components

- React takes that description and puts it on the screen for you

This is the single biggest idea of the day: **you stop telling the browser exactly how to update itself, step by step, and start describing what the finished UI should look like**. React is the tool that turns that description into real HTML on the page.

One thing on purpose is **not** part of today's story: nothing on the page remembers anything yet, and nothing responds to a click or a keystroke. That is real, and it is intentional — it is exactly what

Session 6 adds. Today is only about components and the data you hand them.

2. React vs. TypeScript vs. Vite vs. npm

Four different tools got mentioned today, and it's easy to blur them together. They are not the same thing, and each did a different job.

Tool	What it actually is	What it did for you today
TypeScript	A language — JavaScript plus type-checking. You already know it.	Every <code>.tsx</code> file you wrote is still checked by TypeScript, exactly like Session 4's <code>.ts</code> files were.
React	A <i>library</i> — a set of tools for describing UI as components, and for turning that description into real DOM elements.	It is the thing that reads <code><App /></code> and actually draws <code>Header</code> , <code>StatCard</code> , and <code>TaskItem</code> onto the page.
Vite	A <i>development tool</i> — it runs a local server while you work, and later bundles your files into what a browser can load.	It served your project at <code>npm run dev</code> , and it is the tool that understands <code>.tsx</code> files at all — a browser cannot load <code>.tsx</code> directly.
npm	Node's <i>package manager</i> — installs and manages other people's code (packages) for your project.	It downloaded React, Vite, and everything else your project depends on, and it is what runs the scripts (<code>npm run dev</code>) that Vite defines.

A simple way to keep them straight: **TypeScript is the language you write in. React is the library that builds the UI. Vite is the tool that runs and bundles your project. npm is the tool that installs everything Vite and React need to exist on your computer in the first place.**

You could, in theory, write React without Vite (older tools like Create React App did this differently) — but for this course, Vite is simply the fastest, simplest way to get a real React + TypeScript project running from nothing in one command.

3. Creating the React + TypeScript Project

This section documents exactly what you did, live, at the very start of class — the actual commands, run for real, from the root of your own already-cloned course repository.

Step 1 — create the project:

```
npm create vite@5.5.5 taskflow-react -- --template react-ts
```

Read this command left to right:

- `npm create vite` — asks npm to run Vite's project-creation tool.
- `@5.5.5` — a specific, pinned version of that tool, so everyone in class generates the exact same starting files.
- `taskflow-react` — the name of the folder this command creates.
- `-- --template react-ts` — everything after the first `--` is passed along to Vite itself, and it tells Vite which starting template to generate: React, with TypeScript.

This is **not** something you will run again for Session 6 onward — it happens exactly once, the day a brand-new project is created. From tomorrow, you continue editing this same project.

Step 2 — move into the new folder and install dependencies:

```
cd taskflow-react
npm install
```

`cd taskflow-react` moves your terminal into the folder the previous command just created. `npm install` reads `package.json` (Section 4) and downloads every package your project depends on — React, Vite, TypeScript, and a handful of supporting tools — into a folder called `node_modules`. You will see this folder appear; you never open or edit anything inside it.

Step 3 — start the development server:

```
npm run dev
```

This starts Vite's local server and prints a URL (usually `http://localhost:5173`). Opening that URL in a browser shows the project running live — and because Vite watches your files, saving a change updates the browser automatically, without you refreshing it yourself.

What you should expect to see: the very first time you ran this, the browser showed Vite's own default demo — a spinning logo and a click-counter button, not TaskFlow. That is normal. It is generated boilerplate, not your project, and it was cleaned up live, one small step at a time, before real TaskFlow work began.

We are staying at exactly this level today — what each command does and why you needed it. Deeper npm topics (semantic versioning, exact dependency resolution, publishing packages) are not part of this course.

4. `package.json` — Read Level Only

`package.json` is the file that tells npm (and Vite) everything about your project: its name, the packages it depends on, and the shortcut commands (`scripts`) available to you.

```
{
  "name": "taskflow-react",
  "scripts": {
    "dev": "vite",
    "build": "tsc -b && vite build"
  },
  "dependencies": {
    "react": "^18.3.1",
    "react-dom": "^18.3.1"
  }
}
```

Read only the parts you actually used:

- **dependencies** — the packages your finished app needs to *run*. `react` is the library itself; `react-dom` is the specific piece of React that knows how to put React's output into a real web page's DOM (as opposed to, say, a mobile app — React itself is not tied to the browser, `react-dom` is the part that is).
- **scripts** — named shortcuts. `"dev": "vite"` is why `npm run dev` starts Vite. `"build": "tsc -b && vite build"` is what a real production build does — it type-checks your project with `tsc` first, then asks Vite to bundle it — you will use this more in later sessions, but it's worth recognizing the shape now: check types, then build.

There is a second section, `devDependencies`, listing tools only needed *while building* the project (TypeScript itself, ESLint, Vite's React plugin) — not part of the app that eventually runs in a browser. You do not need to memorize what is in it; recognizing the difference between "needed to run" and "needed to build/develop" is the actual Session 5 takeaway here.

5. The Project File Tree

Here is the real Session 5 End project, simplified to the files that matter:

```
project root/
├─ index.html
├─ package.json
├─ tsconfig.json           (you do not need to edit this)
├─ tsconfig.app.json       (one setting recognized – explained below)
└─ src/
   ├─ main.tsx
   ├─ App.tsx
   └─ index.css
```

```
└─ components/
   └─ Header.tsx
   └─ StatCard.tsx
   └─ TaskItem.tsx
```

The `taskflow-react` name from Section 3 belongs to the project-creation step — it was the folder name Vite created that first day. In your actual course repository, these files live at the repository root; you do not navigate into a nested `taskflow-react/` folder to find them.

"You need to understand and can edit this now": `index.html`, `main.tsx`, `App.tsx`, `index.css`, and every file in `components/`. These are the files real TaskFlow work happens in.

"This exists, but you do not need to understand every setting yet": the `tsconfig*.json` files. You know they configure how TypeScript checks your project — the same idea as Session 4's `tsconfig.json` — but their exact contents (module resolution, build targets, and so on) are configuration detail, not a Session 5 topic. The one exception is `noImplicitReturns`, which was added to `tsconfig.app.json` today purely to keep the same strict TypeScript rule you already know from earlier sessions — nothing new to learn about the file itself.

6. From `index.html` to the Screen

This is the single most important flow to understand today — everything else in this guide is one piece of this picture.

```
index.html
|
|  contains <div id="root"></div>
v
main.tsx
|
|  finds #root, connects React to it
v
App.tsx
|
|  returns the whole page's UI
v
components (Header, StatCard, TaskItem)
|
|  each returns its own piece of UI
v
visible page in the browser
```

`index.html` is almost empty on purpose:

```
<body>
  <div id="root"></div>
  <script type="module" src="/src/main.tsx"></script>
</body>
```

There is exactly one thing inside `<body>` that matters to React: `<div id="root"></div>`, an empty container with no content of its own. The `<script>` tag loads `main.tsx`, and everything you see on the page from that point on was put there by React, not written directly into `index.html`.

7. `main.tsx` — Root Mounting, Line by Line

This is the actual file, unchanged since it was cleaned up live:

```
import { createRoot } from "react-dom/client";
import App from "./App";
import "./index.css";

const rootElement = document.getElementById("root");

if (rootElement) {
  createRoot(rootElement).render(<App />);
}
```

Walk through it exactly as written:

`const rootElement = document.getElementById("root");` — this should look familiar. `document.getElementById` is the same DOM-lookup idea you've used since Session 2, just a different method than `querySelector`. It searches the real page for the element whose `id` is `"root"` — the same empty `<div>` from `index.html` — and gives it back.

`if (rootElement) { ... }` — this is the exact same null-guard discipline from the TypeScript sessions. `getElementById` *can* return `null` if nothing on the page matches, so — exactly like `if (!taskList) { return; }` protected `renderTasks()` in Session 4 — this check makes sure the element genuinely exists before doing anything with it. Nothing about React changes this habit; it's the same TypeScript safety net, applied to a new situation.

`createRoot(rootElement)` — in plain language: *"Set up React to manage everything inside this one element, and only this element."* `rootElement` is React's one connection point to the real page.

`.render(<App />)` — "Now show this component here." `<App />` is not HTML — it looks like a tag because that is TSX syntax (Section 9) for "run the `App` component and put whatever it returns here." Once this line runs, React takes over everything inside `#root`.

Why does `<App />` look like an HTML tag even though `App` is a function? Because that is exactly what TSX is for — a way to write "call this component" using tag syntax instead of a normal function call like `App()`. You will see this same shape everywhere from now on: a capitalized tag name always means "this is one of our own components," never a real HTML element.

8. `import` / `export` — Session 5's Module Story

Every component file needs to be connected to the files that use it. Session 5 needs exactly two shapes of this, and no more.

The shape used by every component you write:

```
function Header() {  
  // ...  
}  
  
export default Header;
```

```
import Header from "../components/Header";
```

`export default Header;` at the bottom of a file says: "if another file imports from this file without asking for something specific by name, give it `Header`." `import Header from "../components/Header";` is the other half — it pulls that exported value into whatever file needs it, under whatever name you choose to give it (usually the same name, by convention).

Why no `{}` around `Header`? Because `Header` is the file's one *default* export — TypeScript already knows exactly what you mean, so no `{}` is needed.

****The one exception — `main.tsx` needs a *named* import:****

```
import { createRoot } from "react-dom/client";
```

The `{}` here means something different: "from this package, give me specifically the thing called `createRoot` — not whatever the package's default export happens to be." A package can export several named things at once; `{}` is how you pick exactly the one(s) you need by name.

Why does a component need to be exported before another file can import it? Because without `export`, a value only exists *inside* the file that defines it — nothing outside that file can see it at all. `export` is what makes a value visible to other files in the first place; `import` is how another file actually reaches in and uses it.

That is the whole module story for today — one default export/import pattern, and one acknowledged named import. A deeper look at ES Modules (multiple named exports, re-exporting, and so on) is not a Session 5 topic.

9. TSX

TSX is TypeScript with one addition: you can write HTML-like markup directly inside your code, and it becomes part of a real value your function can return.

Compare the HTML you already know to the TSX doing the same job:

```
<!-- plain HTML -->
<p>Total Tasks</p>
```

```
// TSX, inside a component
<p className="stat-label">{props.label}</p>
```

Two real differences, both used constantly today:

- **className instead of class**. `class` is a reserved word in JavaScript/TypeScript (used for actual class syntax), so React uses `className` to mean exactly what `class=` means in HTML — same CSS class, different attribute name.
- **`{}` means "stop writing markup for a moment and evaluate a real TypeScript value here."** Anything inside `{}` is ordinary TypeScript — a variable, a property, an expression — and whatever it evaluates to gets inserted into the page at that exact spot.

You saw `{}` used two different ways today:

```
<StatCard label="Total Tasks" value={3} />
```

`label="Total Tasks"` is a plain string — written in quotes, exactly like an HTML attribute, **no** `{}` needed, because it's already text. `value={3}` is a real number — `3`, not `"3"` — so it needs `{}` to tell TSX "this is a TypeScript value, not literal text." Try removing the `{}` around a number sometime and watch TypeScript's own error explain why: without it, `3` would be read as the *text* `"3"`, not the number `3` — and `StatCardProps` says `value` must be a `number`.

One returned root element. Every component this session returns exactly one top-level element (a `<div>`, a `<header>`, a `<li>` — whatever fits), which may itself contain many children. `App`'s outer `<div>` and `TaskItem`'s outer `<li>` are both examples of this rule. For now, every component returns one normal top-level element.

10. Function Components

You already know how to write a normal function. A React component is that same idea, with one addition:

```
a normal function
+
it returns TSX
=====
a React component
```

The simplest real example in the whole project is `Header` :

```
function Header() {
  return (
    <header className="header">
      <h1>TaskFlow</h1>
      <p>All your tasks in one place.</p>
    </header>
  );
}

export default Header;
```

Nothing here is different from any TypeScript function you've written before, except *what it returns*. `Header` takes no parameters and returns a fixed piece of TSX every single time it runs.

Why does `Header` need no props? Because it never shows different data depending on who uses it — the title and tagline are always the same. A component only needs props (Section 12) when different usages of it need to show different information. `Header`'s whole job is simply: "always draw the same page header." That is a complete, legitimate component job on its own.

11. Props — The Mental Model

This is one of the two or three ideas the entire session is built around, so take it slowly.

Props are information a parent component gives to a child component.

```
App
|
| label="Completed"
| value={1}
v
StatCard
```

`App` is the parent here — it renders `StatCard` and decides exactly what data that particular `StatCard` should show. `StatCard` is the child — it receives whatever the parent hands it, and uses it to decide what to display. This is called **parent** → **child data flow**, and the arrow only ever points one way: a parent can hand data down to a child, but a child cannot reach back up and change what its parent is doing. Today, data flows from the parent into the child. That is the only direction you need to understand in this session.

Think of props as arguments to a function, because that is almost exactly what they are — `App` is, in effect, "calling" `StatCard` three separate times with three different sets of arguments, and getting back three differently-filled pieces of UI.

12. Typed Props With Interfaces

You already know `interface` — you used it in Session 4 to describe the shape of API data (`Task`, `User`). Today, the exact same tool describes something new: **the data a component expects to receive**.

```
interface StatCardProps {
  label: string;
  value: number;
}
```

Read this exactly like any other interface: *"anything typed `StatCardProps` must have a `label` that is a `string`, and a `value` that is a `number` — nothing more required, nothing less allowed."* The only thing that's new is *what* it's describing — not a piece of API data this time, but the props one specific component is allowed to receive.

```
function StatCard(props: StatCardProps) {
  return (
    <div className="stat-card">
      <p className="stat-label">{props.label}</p>
      <p className="stat-value">{props.value}</p>
    </div>
  )
}
```

```
        </div>
    );
}
```

`props: StatCardProps` is a typed function parameter — the exact same idea as `userId: number` in Session 4's `getUserName`. TypeScript now checks every place `<StatCard ... />` is used, and refuses to compile if a required prop is missing, misspelled, or given the wrong kind of value (try writing `value="3"` — a string — and watch TypeScript reject it, because `StatCardProps` says `value` must be a `number`).

Why `props.label` and `props.value`? In Session 05 we read properties directly as `props.label` and `props.value`. This is the complete pattern you need today. `props` is one single object, and you reach into it by name, exactly the same way you've always read one property off any other typed object — there is nothing missing here, and nothing you're doing "the hard way."

13. `StatCard` — Complete Walkthrough

`StatCard` is the clearest possible example of the session's central idea, so it's worth seeing the whole story in one place.

The component, combining Sections 11 and 12:

```
interface StatCardProps {
  label: string;
  value: number;
}

function StatCard(props: StatCardProps) {
  return (
    <div className="stat-card">
      <p className="stat-label">{props.label}</p>
      <p className="stat-value">{props.value}</p>
    </div>
  );
}

export default StatCard;
```

All three real usages, from `App.tsx` :

```
<StatCard label="Total Tasks" value={3} />
<StatCard label="Completed" value={1} />
<StatCard label="Pending" value={2} />
```

One component definition. Three completely different results on the page, because each call hands in different props. This is the idea to hold onto for the rest of the course:

```
same component
+
different props
=====
reusable UI
```

You will use this exact recipe — define once, call many times with different data — for every component you build from here on, including the ones in your own homework.

14. `TaskItem` — The Teamwork Component

`TaskItem` was built during teamwork, using exactly the same recipe as `StatCard`, just with more data to describe:

```
interface TaskItemProps {
  title: string;
  ownerName: string;
  statusText: string;
  statusClass: string;
}

function TaskItem(props: TaskItemProps) {
  return (
    <li className="task-item">
      <span className="task-text">
        <span className="task-title">{props.title}</span>
        <span className="task-user">{props.ownerName}</span>
      </span>
      <span className={`task-status ${props.statusClass}`}>
        {props.statusText}</span>
      </li>
    );
}
```

```
}  
  
export default TaskItem;
```

Why four props instead of StatCard 's two? Because a task row genuinely needs to show more information: its title, who owns it, its status as readable text ("Pending"/"Completed"), and a separate CSS class name ("pending" / "completed") used purely for styling the little status pill a different color. `statusText` and `statusClass` look similar, but they do two different jobs — one is what a person *reads*, the other is what CSS *uses*.

One small new bit of JSX syntax appears here: `className={ task-status ${props.statusClass} }`. This is a normal TypeScript template literal (backticks, `${}`) — nothing about it is React-specific — used inside `{}` because, just like any other expression, a template literal is a real TypeScript value that needs `{}`` to appear inside JSX.

All three real usages, from App.tsx :

```
<TaskItem title="Finish JavaScript exercise" ownerName="Leanne Graham"  
  statusText="Pending" statusClass="pending" />  
<TaskItem title="Review pull request" ownerName="Ervin Howell"  
  statusText="Completed" statusClass="completed" />  
<TaskItem title="Write session notes" ownerName="Clementine Bauch"  
  statusText="Pending" statusClass="pending" />
```

We are deliberately not using `.map()` here. Writing out three calls by hand, one per task, is not a shortcut we haven't gotten to yet — it's this session's actual, correct way to render a repeated piece of UI. Rendering a *real, changeable* list from an array using `.map()` is Session 7's job, and it depends on ideas (arrow functions, the `key` prop) that haven't been taught yet. There's no need to apologize for the repetition — it's teaching exactly what it's supposed to: the same component, called several times, with different data each time.

15. App — The Parent That Combines Everything

`App` doesn't invent any new UI of its own. Its entire job is **composition** — combining smaller components into one finished page, and handing each one the props it needs.

```
App  
├ Header (no props — always the same)  
├ StatCard × 3 (label + value)
```

```
| static filter buttons      (plain JSX, no component yet)
| static search input       (plain JSX, no component yet)
└ TaskItem × 3              (title + ownerName + statusText + statusClass)
```

Notice that the filter buttons and search box are **not** their own components yet, even though `Header / StatCard / TaskItem` are — they're still just plain JSX sitting directly inside `App`. That's deliberate, not a missing step: extracting an interactive control into its own component only makes sense once it's actually interactive, and nothing on this page responds to a click or keystroke yet (Section 16). Turning a static control into its own component before it does anything would just be extra ceremony with no payoff yet.

16. Static Controls — On Purpose

The filter buttons and the search box are real, visible, and styled — but right now they do nothing at all:

```
<button className="filter-button active">All</button>
<button className="filter-button">Completed</button>
<button className="filter-button">Pending</button>
```

```
<input type="text" className="search-input" placeholder="Search tasks..." />
```

No `onClick`. No `onChange`. No `useState`. Click any button — nothing happens. Type in the search box — nothing happens, beyond the browser's own default behavior of showing whatever you typed.

This is not broken, and it is not missing code you forgot. It is the exact, intentional Session 5 stopping point. Making these controls actually *do* something requires a component to **remember** something — a click needs to be recorded, typed text needs to be held onto — and nothing in React (or in TSX, or in typed props) gives a component that ability. That missing piece has a name, and it's the very next thing you'll learn: **state**, starting Session 6.

17. CSS Did Not Disappear

React changes how markup gets built. It does not touch CSS at all. The exact same familiar stylesheet approach from every earlier session still applies — `src/index.css` is plain CSS, using plain class selectors:

```
.stat-card {
  flex: 1;
  background-color: #ffffff;
  border: 1px solid #e2e5ea;
  border-radius: 10px;
  padding: 20px;
  text-align: center;
}
```

`className="stat-card"` in a component connects to `.stat-card { ... }` in the CSS file exactly the same way `class="stat-card"` and `.stat-card` would in plain HTML — only the attribute's *name* changed (Section 9), never how CSS selectors work. If you know how to read and write CSS already — and you do — there is nothing new to learn here at all.

18. Complete Session 05 End State

What exists right now:

- `Header` — a component with no props, always drawing the same header.
- `StatCard` — a typed, reusable component, used three times with different `label / value` props (3 total tasks, 1 completed, 2 pending).
- `TaskItem` — a typed, reusable component (built in teamwork), used three times by hand with different `title / ownerName / statusText / statusClass` props.
- Static filter buttons and a static search input — real, styled, and currently inert.
- A React root, mounted once in `main.tsx`, rendering `<App />` into `index.html`'s single `#root` element.
- A working `import / export default` connection between every file.

What is explicitly NOT implemented yet:

- No state anywhere (`useState` has not been taught).
- No event handling (`onClick`, `onChange` do not appear).
- No controlled inputs.
- No `.map()` or any list rendering from an array.
- No arrow functions.
- No `type` aliases or string-literal unions.
- No callback props.
- No data fetching of any kind.

- No object/props destructuring — every prop is read as `props.x`.
- No Fragment — not taught yet.

If a syntax or idea isn't named anywhere above as something you built, you have not missed it — it simply isn't part of Session 5.

19. Common Mistakes

1. Writing `class=` instead of `className=`

WRONG:

```
<header class="header">
```

RIGHT:

```
<header className="header">
```

Why it's wrong: `class` is a reserved word in JavaScript/TypeScript. React uses `className` to mean the same thing HTML's `class` means. TypeScript itself will flag this as an error in TSX.

2. Forgetting `{}` around a non-string prop

WRONG:

```
<StatCard label="Completed" value=1 />
```

RIGHT:

```
<StatCard label="Completed" value={1} />
```

Why it's wrong: `value=1` (no quotes, no braces) is not valid TSX at all. A string prop is written in quotes with no `{}` (`label="Completed"`); anything that is a real TypeScript value — a number, a boolean, an expression — needs `{}`.

3. Passing the wrong type of value for a prop

WRONG:

```
<StatCard label="Completed" value="1" />
```

Why it's wrong: `StatCardProps` says `value: number`. `"1"` in quotes is a `string`, not a `number` — TypeScript will refuse to compile this, the same way it refused `totalCount.textContent = tasks.length;` in Session 4. Read the error message; it will name the exact mismatch.

4. Forgetting `export default`

Why it's wrong: without it, nothing outside the file can import the component at all, even if the function itself is written correctly. The error will point at the `import` line, not the component's own file — that confuses people the first time.

5. A wrong import path

WRONG:

```
import Header from "./Header";
```

RIGHT:

```
import Header from "./components/Header";
```

Why it's wrong: the path must match where the file actually lives on disk. `Header.tsx` lives inside `components/`, so the import path must say so.

6. Reading a prop that isn't in the interface

WRONG:

```
<p>{props.name}</p> // StatCardProps only has label and value
```

Why it's wrong: TypeScript checks `props` against the exact interface you wrote. Reading a property that was never declared is a compile error, not a silent `undefined` the way plain JavaScript would allow.

7. Removing the root-element null check

WRONG:

```
const rootElement = document.getElementById("root");
createRoot(rootElement).render(<App />); // no if-check
```

Why it's wrong: `getElementById` can return `null`. `createRoot` needs a real element, not `null`. This is the exact same discipline as any DOM null-guard from the TypeScript sessions — don't drop it just because the surrounding code is now React.

8. Trying to make the controls interactive already

Why it's wrong: not a compile error, but a sign of reasoning ahead of the lesson. `onClick`, `onChange`, and `useState` are genuinely next session's material — trying to wire them in today usually means copying syntax from somewhere without yet knowing what it does. It's fine to be curious about it; it's better to wait one session and build it with real understanding.

20. How to Read the App When You're Lost

A practical order for finding your way through any React project when you're not sure where something lives:

1. **index.html** — is there really just one `<div id="root">`? Nothing else should be drawing the page.
2. **main.tsx** — confirm the root is found, guarded, and `<App />` is rendered into it.
3. **App.tsx** — this is the top of the whole component tree. Everything the page shows is reachable by reading down from here.
4. **Find the component's import** — every tag like `<StatCard ... />` has a matching `import` line near the top of the same file. That import line tells you exactly which file to open next.
5. **Open the component's own file** — read what it returns, and what props it expects.
6. **Check the props interface** — this tells you exactly what data the component needs, without reading a single line of its actual logic.
7. **Check the CSS class** — if something *looks* wrong but the data seems right, the bug is probably in `index.css`, not in the component.

Following this order, in this direction, will get you to almost any bug in a small React project without needing to read the whole codebase first.

21. Glossary

React A library for describing UI as components, which then puts that description onto a real web page.

Vite A development tool that runs your project locally while you work, and bundles it for production.

npm Node's package manager — installs and manages the packages (like React and Vite) your project depends on.

component A normal function that returns JSX — a reusable, self-contained piece of UI.

TSX TypeScript with the added ability to write HTML-like markup directly inside your code.

prop A single piece of information a parent component passes down to a child component.

parent / child In a `<Parent><Child /></Parent>` relationship, the parent renders the child and decides what props to give it. Data only ever flows parent → child, never the other way, in what Session 5 covers.

interface (as used for props) A named description of exactly which props a component expects, and what type each one is.

import / export default The pair of keywords that connect one file's value to another file that needs to use it.

named import An import using `{}`, for pulling one specifically-named value out of a package that may export several — for example `import { createRoot } from "react-dom/client";`.

root (element) The one real DOM element, `<div id="root">`, that React is given control over.

createRoot(...).render(...) The two-step call that connects React to the root element, then draws a component into it.

className React's name for the HTML `class` attribute — connects an element to a CSS rule, exactly like `class` always has.

22. Review Questions

1. What actually changed today — the language you write in, or how the UI gets built?
2. What is the difference between React, Vite, and npm?
3. Why does `Header` not need any props?
4. Why does `StatCard` need props?
5. What does `value={3}` mean, and why does it need `{}` when `label="Total Tasks"` doesn't?

6. Why does `main.tsx` check `if (rootElement)` before calling `createRoot` ?
 7. Why do we write `className` instead of `class` ?
 8. Why are `StatCard` and `TaskItem` each written once but used multiple times?
 9. Why are the three `TaskItem` s written out by hand today, instead of using `.map()` ?
 10. Why do the filter buttons and search box currently do nothing when clicked or typed in?
 11. What is the one named import this session actually needs, and why does it use `{}` ?
-

23. Answers

1. How the UI gets built. You are still writing TypeScript. What's new is describing UI with components instead of manually finding and changing DOM elements yourself.
2. React is the library that builds UI out of components. Vite is the development/build tool that runs your project and understands `.tsx` files. npm is the package manager that installs React, Vite, and everything else your project depends on.
3. Because it never needs to show different data — the title and tagline are always the same, no matter where `Header` is used. Props only matter when different usages need different data.
4. Because it's used three times on the same page with three genuinely different values (label and value). Without props, there would be no way for one `StatCard` definition to show three different results.
5. It means "put the real TypeScript value `3` here." It needs `{}` because `3` is a number, not text — without `{}`, TSX would treat it as the literal characters `"3"`, and `StatCardProps` requires a real `number`. `"Total Tasks"` is already text, so no `{}` is needed.
6. Because `document.getElementById("root")` can return `null` if nothing on the page matches — exactly the same DOM null-guard discipline used throughout the TypeScript sessions, just applied to React's own root element this time.
7. Because `class` is a reserved word in JavaScript/TypeScript syntax (used for real class definitions), so React needed a different name for the same idea — `className` does exactly what `class=` does in plain HTML.
8. Because each one describes a *shape* of UI, and props let that same shape display different data every time it's used — "same component, different props" is what makes a component reusable at all.
9. Because `.map()` — the tool for turning a real array into repeated UI — hasn't been taught yet. It depends on arrow functions and the `key` prop, both genuinely new ideas that belong to Session 7. Three manual calls is the correct, complete way to do this with what Session 5 actually teaches.

10. Because nothing wires them to any behavior yet — no `onClick`, no `onChange`, and no `useState` to remember anything even if there were. That is intentional; it's exactly the gap Session 6 fills.

11. `{ createRoot }` from `"react-dom/client"`, used in `main.tsx`. The `{}` means "give me specifically the thing named `createRoot`, not whatever this package's default export happens to be" — a package can have several named exports, and `{}` picks exactly the one you ask for by name.

24. What You Should Be Able to Explain Now

- ☐ I can explain the difference between React, Vite, and npm.
- ☐ I can explain how `index.html` eventually reaches `App.tsx`.
- ☐ I can explain what `createRoot(...).render(<App />)` does, and why the null check exists.
- ☐ I can explain what a component is, in my own words.
- ☐ I can write a normal function component with no props.
- ☐ I can write a typed props interface and use `props.x` to read from it.
- ☐ I can explain parent → child data flow.
- ☐ I can import and export a component correctly.
- ☐ I can explain why `TaskItem` is written three times by hand, and why that's correct for now.
- ☐ I can explain why the filter buttons and search box are still static.

If every box is honestly checked, you understand Session 5 completely — and you're ready for the one idea that's missing: a component that can remember something on its own. That's Session 6.